EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**Splitting work means the components never need to communicate.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.





DO NOW — RECALL

# Lesson 066 · Problem Decomposition: Build a Useful Solu

Name: _____ Class: _____ Date: _____

**1. What makes a problem suitable for computation?**

**2. What is an abstraction?**

**3. What is a procedure?**

**4. Stack initially empty: Push A, B, C; Pop; Push D. Give the stack top-to-bottom.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

Explicit inputs, constraints and a testable outcome.

### 2. Model answer

A representation keeping relevant detail and omitting irrelevant detail.

### 3. Model answer

Named subroutine performing a task.

### 4. Model answer

D, B, A. C is popped and D is pushed above B.

### By the end, you can…

• Decomposition divides a larger problem into manageable sub-problems whose inputs and outputs are clear.

• A top-down design begins with a main task, then refines it into independently testable functions or procedures.

• Interfaces between subproblems matter: validation should happen before data are stored or a fee is charged.

### Vocabulary — know these terms

decomposition — Split task into smaller problems.

top-down design — Start broad, refine into subtasks.

interface — Inputs/outputs between components.

modularity — Small cohesive program components.





YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

Explain two benefits and one limitation of decomposition when a team builds a school booking app. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Identify three independent subtasks and their outputs for a library fine system. [6]

### Question 2

Write C# Fine(daysLate) for 20p/day capped at £2, zero for on-time returns. [6]





NEW LEARNING

# Read, annotate and explain

### Define and predict

Decomposition divides a larger problem into manageable sub-problems whose inputs and outputs are clear.

### Explain

A top-down design begins with a main task, then refines it into independently testable functions or procedures.

### Evaluate

Interfaces between subproblems matter: validation should happen before data are stored or a fee is charged.

**Explain one step in your own words, using the correct technical terms.**

I DO — WORKED EXAMPLE

# Follow the worked method

**Decompose a cinema price calculator for child/adult tickets.**

**Follow the reasoning**

State the interface for each subtask; every step has a specific responsibility.





WE DO — GUIDED PRACTICE

# Try → Discuss → Improve